¿Cuándo conviene repartir el trabajo?

En la etapa anterior mejoramos el acceso a memoria de right-looking. Ahora damos el primer paso hacia memoria distribuida: implementamos MPI en C y comparamos propiedad estática contigua y cíclica. Python analiza los tiempos y prepara gráficos; no realiza la factorización.

Hipótesis inicial

Con matrices pequeñas, comunicar cada columna puede costar más que el cálculo ahorrado. Al crecer n, esperamos que varios procesos resulten útiles. El reparto cíclico debería equilibrar mejor las actualizaciones, pero eso **no garantiza** que sea el más rápido en esta implementación.

Qué hace cada proceso

1. Asignar: repartimos las columnas antes de factorizar. Cada proceso guarda solo las suyas, empaquetadas por filas.
2. Calcular: el dueño de la columna j calcula el pivote y los elementos inferiores de esa columna.
3. Difundir: todos llaman a MPI_Bcast; el dueño envía el tramo de la columna desde j hasta n−1.
4. Actualizar: cada proceso modifica las posiciones de sus columnas k > j, usando el vector recibido. Luego avanzamos a j+1.

$$a_{ik} \leftarrow a_{ik} - l_{ij} l_{kj}, \qquad i \geq k > j$$

Los destinos pertenecen a procesos distintos: las actualizaciones de una etapa son independientes. Las columnas sucesivas siguen teniendo dependencias. Usamos comunicaciones bloqueantes y no incorporamos solapamiento ni tareas dinámicas.

La misma matriz, dos propietarios posibles

n=8, P=3	Columnas de P0	Columnas de P1	Columnas de P2
Contiguo	0, 1, 2	3, 4, 5	6, 7
Cíclico	0, 3, 6	1, 4, 7	2, 5

En contiguo, las primeras n mod P tareas reciben una columna adicional. En cíclico, owner(k)=k mod P. Incluso si P > n, los procesos sin columnas participan en las comunicaciones. El reparto no cambia durante la factorización.

MPI_Bcast distribuye un mensaje desde el dueño a todos los procesos del comunicador. Fuente: documentación oficial de Open MPI.

Cómo medimos y qué comparamos

Decisión	Protocolo ejecutado
Equipo	Un equipo, WSL2 / Ubuntu 26.04, AMD Ryzen 5 2600; Open MPI 5.0.10 y GCC 15.2.0.
Compilación	-O3 -std=c11 -Wall -Wextra -Wpedantic; double, sin BLAS.
Tamaños y procesos	n = 64, 256, 512, 1024; P = 1, 2, 3, 4. Un proceso por núcleo distinto, según la topología expuesta por WSL.
Repeticiones MPI	Un calentamiento por reparto y 8 repeticiones medidas. Contiguo y cíclico alternan el orden. El orden de P rota entre tamaños.
Referencias secuenciales	Left, right original y right con vector. Seis repeticiones antes y seis después de MPI para cada tamaño; misma semilla 2026.

El tiempo principal incluye cálculo y comunicación

$$T_P = \max_{r=0,\ldots,P-1} (t_{r,\text{fin}} - t_{r,\text{inicio}})$$

Una barrera coordina el inicio. Cada proceso mide su duración con MPI_Wtime; al finalizar tomamos el máximo. No restamos marcas de reloj de procesos diferentes. El cronómetro incluye el armado del vector, las difusiones y las actualizaciones; excluye generación, reserva, distribución inicial, reunión del factor y comprobaciones. Tampoco incluye iniciar mpirun.

Por tanto, medimos el kernel de factorización, no el tiempo de la aplicación completa. El proceso 0 genera y reúne matrices completas para este banco de pruebas; esa preparación también limita la escalabilidad futura.

Dos aceleraciones responden preguntas distintas

$$S_{\text{MPI}} = \frac{\widetilde{T}_{\text{MPI},1}}{\widetilde{T}_{\text{MPI},P}}, \qquad S_{\text{util}} = \frac{\min_v \widetilde{T}_{v,\text{seq}}}{\widetilde{T}_{\text{MPI},P}}$$

La primera compara un reparto consigo mismo al aumentar P. La segunda comprueba si supera al mejor secuencial de esta campaña; esa es la relevante para decidir si nos conviene usar MPI. Reportamos medianas y cuartiles inclusivos. Los cuartiles describen dispersión; no son intervalos de confianza.

MPI_Wtime mide tiempo local. Fuente: documentación oficial de Open MPI.

Resultado: más procesos no siempre ayudan

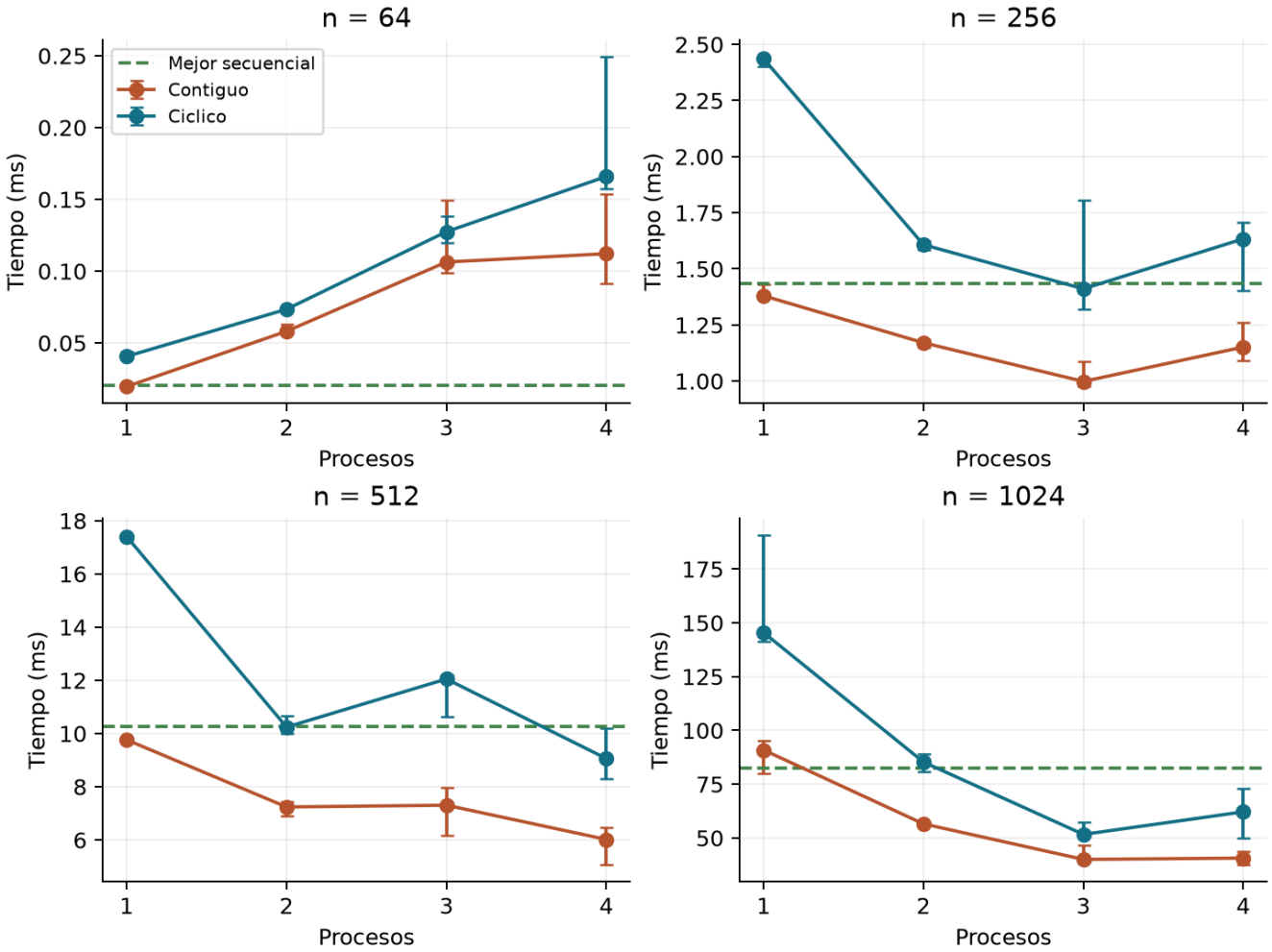


Figura 1. Medianas de 8 repeticiones por configuración; barras Q1-Q3. La línea verde muestra la mejor mediana secuencial contemporánea. Cada panel tiene su propia escala vertical.

Lo que vemos

n=64: usar más de un proceso aumenta el tiempo en ambos repartos. Hay demasiado poco trabajo para amortizar la coordinación.
n=256 y n=512: el contiguo ya obtiene mejoras útiles; el cíclico presenta resultados cercanos al secuencial o más lentos en varias configuraciones.
n=1024: ambos repartos mejoran al pasar de 1 a 3 procesos. Agregar un cuarto no reduce la mediana en esta campaña.

Contiguo y cíclico comparten el código de factorización y las operaciones matemáticas. Sin embargo, el reparto cambia los índices, los accesos al vector difundido y el código que puede optimizar el compilador. Esta prueba compara las implementaciones completas, no aísla exclusivamente el balance de carga.

¿Cuándo empieza a convenir MPI?

n	Mejor sec. (ms)	Mejor MPI P≥2	MPI (ms)	S útil
64	0,021	Contiguo, P=2	0,058	0,35
256	1,434	Contiguo, P=3	0,998	1,44
512	10,265	Contiguo, P=4	6,019	1,71
1024	82,654	Contiguo, P=3	40,028	2,06

Selección descriptiva de la menor mediana entre configuraciones probadas, no una garantía de que siempre sea la mejor. Right con vector fue la mejor referencia secuencial en los cuatro tamaños.

Respuesta provisional para este equipo

Con n=64 no conviene paralelizar este kernel. El primer tamaño probado con mejora clara del contiguo es n=256. Para n=1024 y P=3, el contiguo tarda 40,03 ms frente a 82,65 ms del secuencial: S útil = 2,06.

No encontramos un umbral exacto. Faltan tamaños intermedios y campañas independientes. Tampoco suponemos que la conveniencia sea monótona: caché, reparto, sistema operativo y red pueden cambiar el resultado. P=3 y P=4 contiguos tienen cuartiles que se superponen en n=1024; no declaramos una superioridad concluyente de tres procesos.

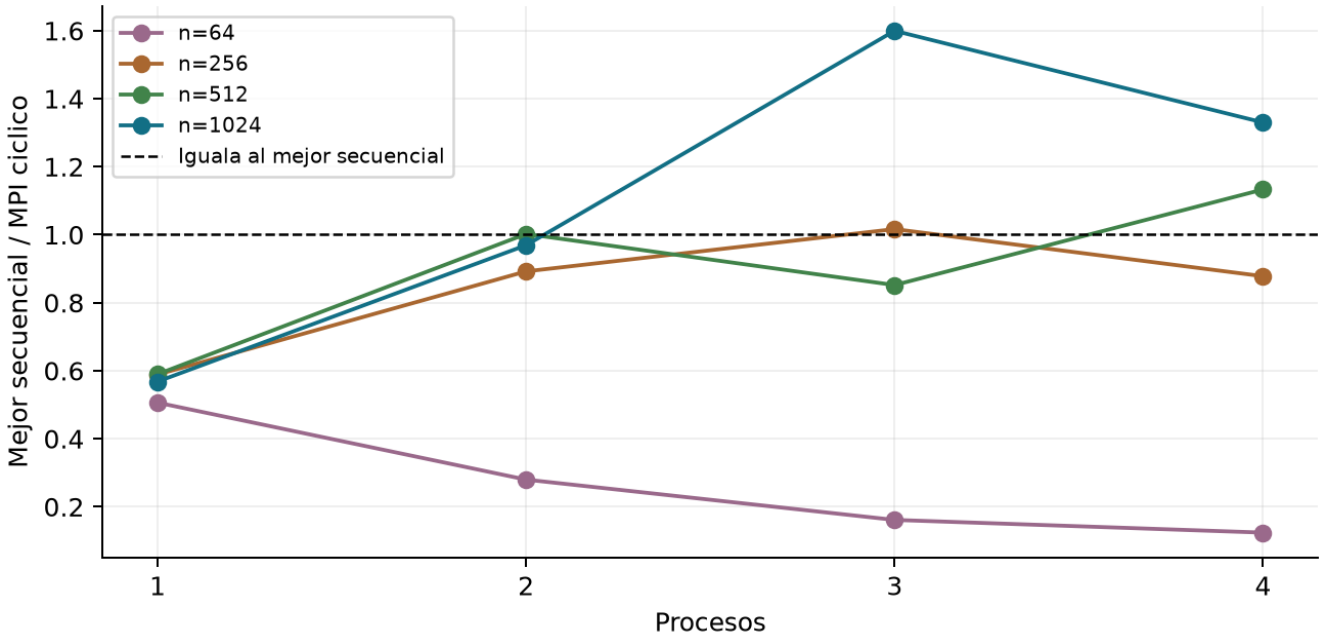


Figura 2. Aceleración útil del cíclico. Cerca de 1, una pequeña diferencia de medianas no basta para afirmar una mejora robusta. La evaluación entre nodos queda pendiente.

¿Por qué no ganó el cíclico?

La hipótesis de mejor reparto sí se cumple para el **número de actualizaciones**. Para n=1024 y P=3, los conteos exactos por proceso son:

Reparto	P0	P1	P2
Contiguo	46 434 993	86 086 814	46 434 993
Cíclico	59 652 153	59 652 153	59 652 494

Pero el tiempo depende también de cuánto cuesta cada actualización y cuándo queda disponible. En el contiguo, el vector difundido se lee en posiciones consecutivas. En el cíclico, sus índices avanzan de P en P, aunque los destinos locales estén empaquetados. El cálculo de índices también difiere.

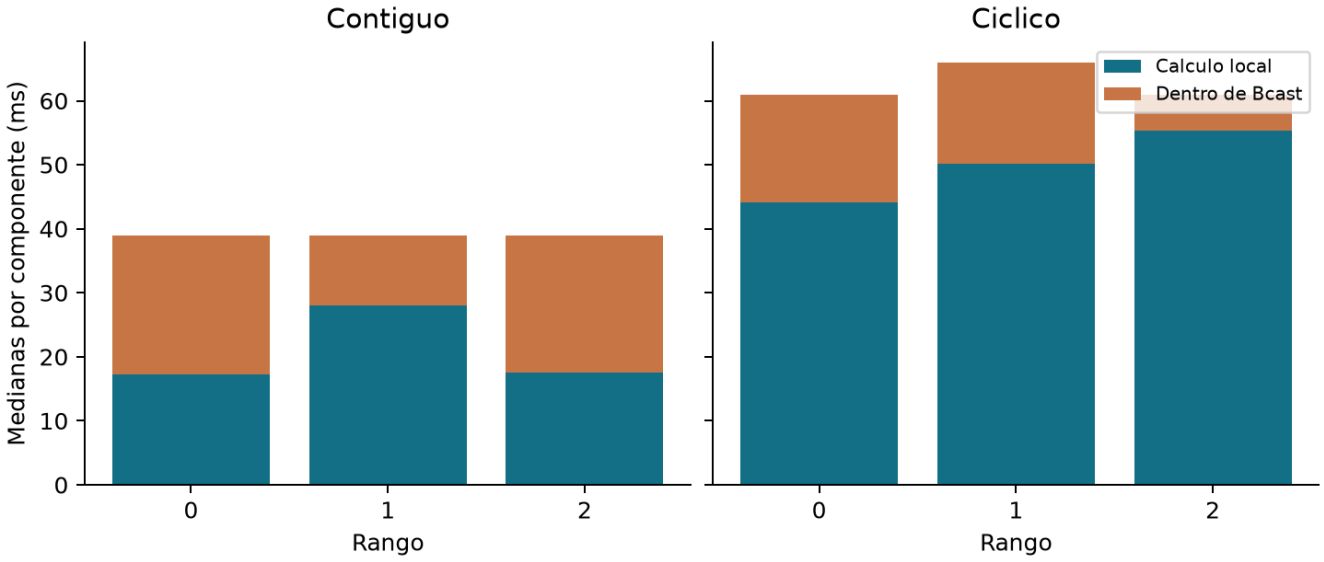


Figura 3. Perfil separado para n=1024, P=3, con 3 repeticiones e instrumentación interna. Se apilan medianas de componentes; su suma no es necesariamente la mediana del total. Bcast incluye transferencia y espera por otros procesos: no mide únicamente red.

Un control que impide una conclusión apresurada

Con P=1, ambos repartos poseen las mismas columnas. Aun así, el contiguo tarda 90,90 ms y el cíclico 145,49 ms. Por tanto, **el desequilibrio entre procesos no explica toda la diferencia**. Los caminos de indexación y optimización local ya tienen un costo distinto.

El informe de GCC registra versiones vectorizadas y no vectorizadas del bucle de actualización. Es una pista compatible con la diferencia de costo local, pero no cuantifica su causa. No medimos contadores de caché ni aislamos cada optimización del compilador.

Qué aprendimos y cómo seguimos

Decisión respaldada por esta campaña

Ya tenemos una implementación MPI correcta con dos repartos estáticos. En los tamaños probados, el contiguo obtuvo menores medianas que el cíclico para cada P. Lo conservamos como referencia práctica; mantenemos el cíclico como candidato por su mejor equilibrio de operaciones. No elegimos una distribución de forma definitiva para el clúster.

¿Y si ambos leyeran un vector local contiguo?

Esa es la próxima pregunta. Después de recibir la columna, cada proceso podría copiar los valores que necesita a un vector ordenado como sus columnas locales. El bucle interno leería posiciones consecutivas tanto para contiguo como para cíclico. La copia costaría trabajo adicional: debemos incluirlo en el tiempo y comprobar si se amortiza.

Este experimento permitiría reducir la diferencia de acceso e indexación antes de atribuir los resultados al reparto. Después repetiríamos el barrido de n y P en el clúster, distinguiendo un nodo de varios nodos. Todavía no incorporamos bloques, reparto dinámico ni comunicación solapada.

Comprobaciones realizadas

Ambos repartos pasaron casos SPD de tamaños 1, 2, 7, 16 y 31, un factor conocido 3×3 y rechazo colectivo de pivotes negativos y NaN, con P=1,2,3,4. Incluimos repartos desiguales y P > n. Las 256 mediciones MPI tienen residuo relativo ≤ 10^{-12} y coincidieron con el factor right secuencial dentro de 10^{-13}; el residuo máximo observado fue $1{,}2131\times10^{-15}$.

UndefinedBehaviorSanitizer pasó con P=4. AddressSanitizer también pasó con P=4 al desactivar memory/patcher de Open MPI solo en la prueba. Con ese componente activo fallaba dentro de MPI_Init, incluso en un programa mínimo sin Cholesky. No se comprobó detección de fugas; las mediciones usaron compilación normal y la configuración MPI predeterminada.

Archivos para reproducir y revisar

Archivo en desde_cero	Contenido
src/cholesky_mpi.c	Propietarios, índices y factorización distribuida.
src/compare_mpi.c	Banco de pruebas, validación, tiempo y perfil.
scripts/comparar_mpi.sh	Campaña con 1, 2, 3 y 4 procesos.
scripts/analizar_mpi.py	Estadísticas, aceleración, cuartiles y gráficos.
resultados/mpi_20260921/	CSV, entorno, afinidad, SHA-256 y resúmenes.

Los 144 tiempos secuenciales, 256 tiempos MPI y 60 registros de perfil por rango se conservan por separado. Las etapas anteriores permanecen en el PDF de fundamentos; este capítulo continúa su análisis.